

ИТМО · ТЕСТИРОВАНИЕ ПО · ОСЕНЬ 2026

Таблицы решений, state-transition, дефекты и баг-репорты

Модуль 2 · Лекция 4 · Тест-дизайн

Ольга Лихачева · Lekton

4 курс

24 сентября · Четверг 13:30–15:00

Баг-репорты: раздел лекции

План лекции

- 01** Как выбрать технику под задачу
- 03** Диаграммы и таблицы состояний/переходов (state-transition)
- 05** Баг-репорты: структура, severity vs priority, реальный пример

- 02** Таблицы решений (decision tables) — сквозной пример на клиенте
- 04** Дефекты: определение, классификация, жизненный цикл

Исходные предпосылки: результаты лекции 3

Освоенные техники и их правила

Классы эквивалентности. Разбиение входных значений по признаку одинаковой обработки.

Правило: один представитель на класс.

Граничные значения. Точки на границах классов и в соседних классах. Правило: три точки на границу.

Попарное тестирование. Покрытие всех пар значений параметров. Правило: строка набора — тест-кейс.

Класс задач, не покрываемых базой

Комбинации условий. Решение определяется совместным выполнением нескольких независимых условий, а не значением отдельного поля.

Последовательность операций. Результат зависит от текущего состояния объекта и от предшествующих событий.

Положения лекции 3, используемые далее как правила: невалидные значения не объединяются в одном тест-кейсе; на каждую границу берётся три точки; при отсутствии правила в спецификации вопрос адресуется аналитику.

Критерии выбора техники тест-дизайна

Характер зависимости результата	Техника	Уровень применимости
Результат определяется значением поля или его положением относительно границы	Классы эквивалентности и граничные значения	База, применима к любому объекту
Результат определяется совместным выполнением нескольких условий	Таблицы решений	Комбинации условий, дискретные действия
Результат зависит от текущего состояния и порядка операций	Тестирование переходов состояний	Объекты с жизненным циклом
Результат зависит от сочетаний нескольких параметров, зависимости неизвестны	Попарное тестирование	Много параметров, полный перебор невозможен

Сквозной пример лекции: заведение клиента

Один объект на всю лекцию: `POST /api/v1/clients/create-individual` — тот же запрос, что на лекции 3.

Поля запроса

`contacts.email, contacts.phone`
`individual.demographic.birth_date`
`individual.demographic.gender`
`individual.demographic.marital_status`
`tax.tax_number, external_id`

Что показывает каждая техника

Таблица решений — от чего зависит код ответа при создании.

State-transition — жизненный цикл клиента: создан, активен, заблокирован.

Баг-репорт — как описать найденный дефект.

Один объект связывает все разделы лекции и позволяет сопоставлять техники на общих данных.

МОДУЛЬ 2 · ЛЕКЦИЯ 4

Таблицы решений

Таблицы решений: определение и структура

Таблица решений — компактная запись сложных бизнес-правил.

Условия — верх

Набор условий, от которых зависит решение.

Действия — низ

Что система должна сделать.

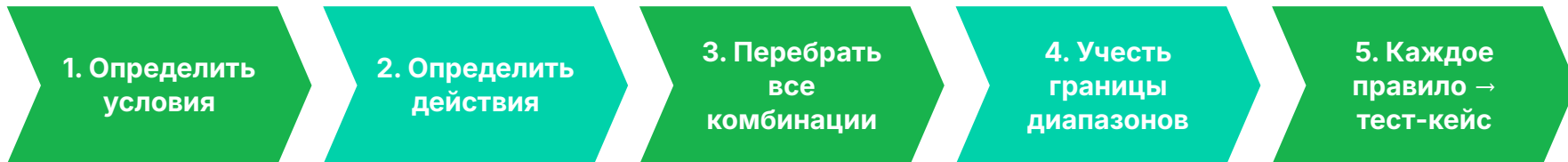
Правила — столбцы

Каждое правило = уникальная комбинация условий → набор действий.

Действия зависят **только от значений условий**, не от порядка.

Из таблицы напрямую получаются тест-кейсы.

Таблицы решений: методика построения



Комбинаторика

Для n бинарных условий — 2^n правил. Условия бывают бинарные или диапазонные.

Диапазонные условия

Учесть границы диапазонов — таблица решений комбинируется с BVA.

Ограничение: рост экспоненциальный. Больше пяти-шести условий — таблица становится громоздкой.

Таблицы решений: расчёт числа правил

Число правил полной таблицы: 2^n , где n — количество бинарных условий.

Бинарных условий (n)	Правил в полной таблице	Пример лекции
2	$2^2 = 4$	Автострахование: брак × статус студента
3	$2^3 = 8$	Заказ: идентификатор × количество × средства
4	$2^4 = 16$	Клиент: обязательные поля × форматы × возраст × external_id
5–6	32–64	Практический предел читаемости таблицы

Вывод: каждое условие принимает 2 значения, поэтому каждое новое условие удваивает число комбинаций: $2 \times 2 \times \dots \times 2$ (n раз). Для диапазонных условий множитель равен числу значений условия.

Ограниченный вход

Условие принимает значения «Да» и «Нет». Правило задаётся комбинацией этих значений.

Расширенный вход

Условие принимает перечислимый набор значений: «младше 18 / 18–65 / старше 65».

Полная и сокращённая

Полная перечисляет все комбинации. Сокращённая объединяет правила по безразличным условиям.

Применение при реализации

Сокращённая форма: меньше ветвей, компактнее код.

Применение при тестировании

Полная форма: каждая комбинация проверяется, предположения не принимаются.

Помета «НВ» (не важно) в сокращённой форме выражает **предположение** о безразличности условия. Проверка такого правила предположение не подтверждает.

Таблицы решений: пример 1 — автострахование

Условия: «состоит в браке?», «хороший студент?» (Да/Нет). Два бинарных условия → $2^2 = 4$ правила.

(Да, Да)

Скидка 60%

(Да, Нет)

Скидка 25%

(Нет, Да)

Скидка 50%

(Нет, Нет)

Скидка 0%

Каждое правило → тест-кейс с ожидаемым действием.

Таблицы решений: пример 2 — заказ на покупку

Условия: «верный идентификатор?», «верное количество?», «достаточно средств?». Три бинарных условия → $2^3 = 8$ правил. Купить можно, только если все три — «Да».

Схлопывание

Значение «не важно» (NB): если ID неверный — остальные условия неважны, правило одно.

Для тестов

Схлопнутая таблица удобна для разработки, но для тест-кейсов берём **несхлопнутую**.

Оптимизация таблицы сама может внести ошибки — используйте полную таблицу как основу тестов.

Сквозной пример: таблица решений для клиента

Условия создания клиента и ожидаемый результат

Условие	Возможные значения
Обязательные поля заполнены	Да / Нет
Форматы полей корректны	Да / Нет
Возраст по birth_date ≥ 18	Да / Нет
external_id не дублируется	Да / Нет

Четыре бинарных условия $\rightarrow 2^4 = 2 \times 2 \times 2 \times 2 = 16$ правил. Действие одно из трёх: 201 Created, 400 VALIDATION_ERROR, 409 DUPLICATE.

Для тестов берём **несхлопнутую** таблицу: даже если поле не заполнено, проверяем, как система реагирует на остальные условия.

Сквозной пример: разбор правил клиента

Обязат.	Форматы	≥ 18	external_id	Ожидаемый результат
Да	Да	Да	уникален	201 Created, клиент создан
Да	Да	Да	дубликат	409 DUPLICATE
Да	Да	Нет	уникален	400, код AGE_LIMIT
Да	Нет	Да	уникален	400, код VALIDATION_ERROR
Нет	—	—	уникален	400, REQUIRED_FIELD

Каждая строка — **тест-кейс** с ожидаемым кодом и телом ответа.

Последняя строка — пример **схлопывания**: без обязательных полей остальные условия неважны.

Таблицы решений: источники условий и действий

Условия выводятся из

Требований. Правило поведения, описанное в спецификации.

Классов эквивалентности. Множество допустимых значений условия.

Границ диапазонов. Переходы между диапазонными значениями.

Действия выводятся из

Реакций системы. Код ответа, изменение состояния, запись в хранилище.

Классов выходных значений. Группировка ответов системы.

Диапазонное условие раскрывается границами: условие $\text{возраст} \geq 18$ проверяется тройкой {17, 18, 19}.

Таблицы решений: применение и ограничения

Применять

Сложные бизнес-правила.

Комбинации условий с дискретными действиями.

Решения, где важна полнота перебора.

Не применять

Система без правил-комбинаций — таблица вырождается.

Более 5–6 условий (32–64 правила) — таблица становится нечитаемой.

Поведение зависит от порядка — это state-transition.

СМП: таблица решений авторизации = статус карты × лимиты × баланс → код ответа.

МОДУЛЬ 2 · ЛЕКЦИЯ 4

State-transition

State-transition: термины



Состояние

Период ожидания события, в котором система хранит предысторию



Переход

Изменение состояния по событию



Событие

Триггер перехода — внешнее или внутреннее (например, таймер)

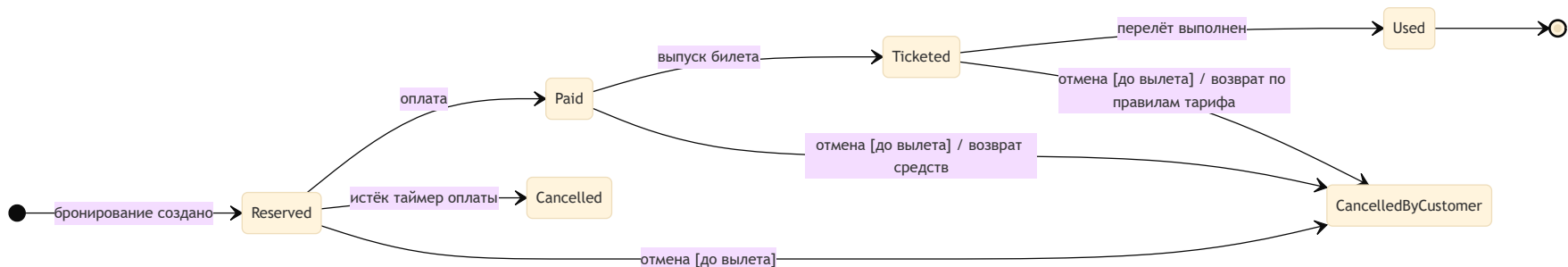


Действие

Операция, вызванная переходом — записывается после «/»

Нотация: чёрная точка — старт, «бычий глаз» — конечное состояние.

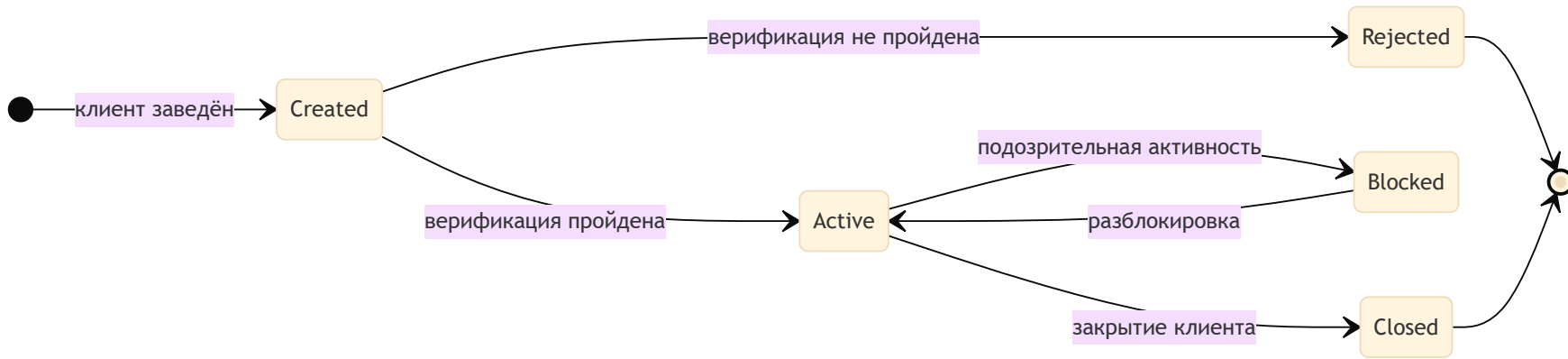
State-transition: диаграммы (пример бронирования)



Условие-охранник (guard) — условие, которое должно быть истинно для выполнения перехода; записывается в квадратных скобках: событие [условие] / действие .

Одна диаграмма — один объект: не смешивать бронирование и пассажира.

Сквозной пример: жизненный цикл клиента



Состояния: **Created / Active / Blocked / Closed** (+ Rejected).

Главная цель техники — **неправильный порядок операций**.

State-transition: таблицы переходов

Таблица из четырёх колонок: Текущее состояние | Событие | Действие | Следующее состояние.

Перечисляет **все** пары «состояние × событие», включая недопустимые.

Польза: помогает найти «дыры» в требованиях до написания кода.

Недостаток: таблица быстро растёт — много пустых клеток.

Пример: «Created + оплата» — недопустимая пара, о которой требования часто молчат.

Сквозной пример: таблица переходов клиента

Текущее состояние	Событие	Следующее состояние	Ожидание
Created	верификация пройдена	Active	разрешено
Created	открыть счёт	Created	отказ, код NOT_VERIFIED
Blocked	разблокировать	Active	разрешено
Active	разблокировать	Active	отказ, код INVALID_STATE
Closed	любое	Closed	отказ, код CLIENT_CLOSED

Строки 2, 4 и 5 — **недопустимые переходы**. Каждая — негативный тест-кейс, который часто отсутствует в требованиях.

Переходы состояний: диаграмма и таблица

Диаграмма состояний

Отвечает на вопрос, какие состояния существуют и как между ними переходят.

Компактна и наглядна, удобна для согласования модели.

Не отображает недопустимые пары «состояние × событие».

Таблица переходов

Отвечает на вопрос, что происходит при каждом сочетании состояния и события.

Перечисляет все пары, включая недопустимые.

Размер равен произведению числа состояний на число событий.

Обе формы описывают **один и тот же автомат**. Диаграмма применяется для обзора и согласования, таблица — для вывода тест-кейсов и поиска пробелов в требованиях.

State-transition: уровни покрытия

1

Уровень 1 — все состояния

Каждое состояние посещено хотя бы раз (низкий уровень)

2

Уровень 2 — все события

Каждое событие выполнено хотя бы раз

3

Уровень 3 — все пути

Может быть бесконечно из-за петель

4

Уровень 4 — все переходы

Рекомендуемый уровень покрытия

Для критичных систем дополнительно проверяем **недопустимые переходы**.

Переходы состояний: пути, циклы, недостижимость

Путь

Последовательность переходов от начального состояния. При наличии циклов множество путей неограниченно.

Цикл

Возврат в ранее пройденное состояние. Требуется проверки повторного входа и повторяемости операций.

Недостижимое состояние

Состояние, в которое не ведёт ни один переход. Признак ошибки в модели или отсутствующей ветви в реализации.

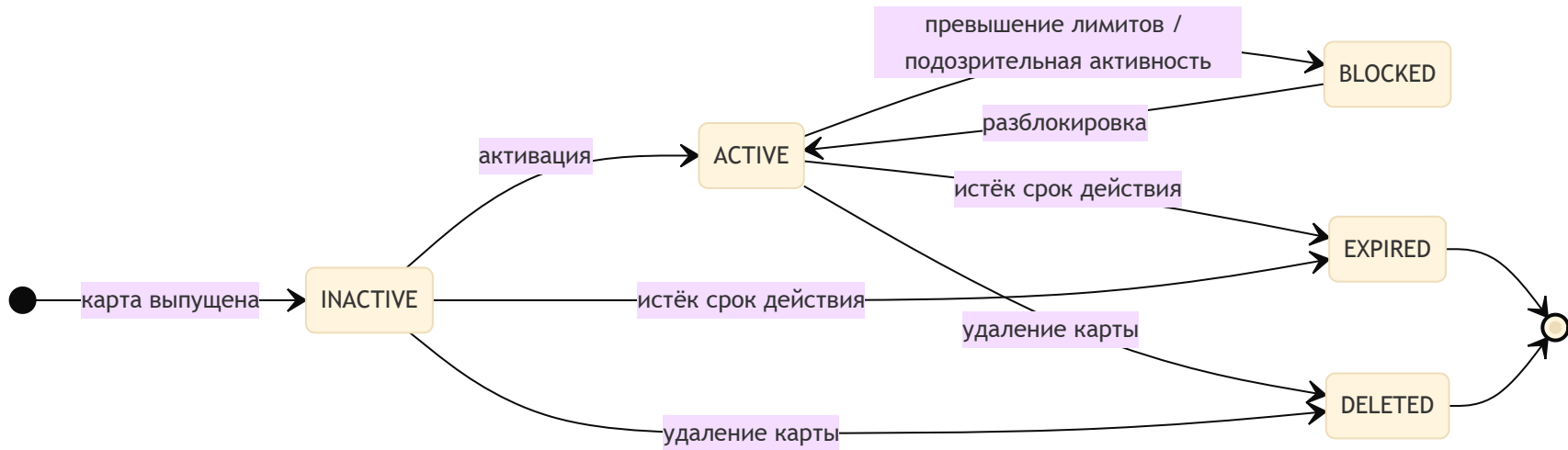
Проверка циклов на СМП

Цикл ACTIVE → BLOCKED → ACTIVE: повторная блокировка и разблокировка не должны искажать состояние карты.

Проверка недостижимости

Если состояние EXPIRED недостижимо ни одним событием, требования и реализация расходятся.

State-transition в СМП: статусы карты



Статусы: **ACTIVE / INACTIVE / BLOCKED / EXPIRED (+ DELETED)**.

Неправильный порядок операций — главная цель техники.

State-transition: применение и ограничения

Применять

Поведение зависит от состояния и порядка операций.

Есть жизненный цикл объекта: клиент, карта, заказ.

Нужно найти дыры в требованиях.

Не применять

Поведение не зависит от предыстории.

Результат определяется только комбинацией условий
— это таблица решений.

Состояний и событий слишком много.

Правило выбора: если порядок операций меняет результат — state-transition; если порядок не важен, а важна комбинация — таблица решений.

МОДУЛЬ 2 · ЛЕКЦИЯ 4

Дефекты

Дефект: определение и смежные понятия

Дефект (defect) — несоответствие фактического результата работы системы ожидаемому результату, заданному требованиями или иным источником ожиданий (ISTQB Glossary).

Ошибка

Действие человека, приведшее к неверному результату.
Синоним в стандартах — mistake.

Дефект

Несовершенство в артефакте: в коде, требовании, документе.
Синонимы — fault, bug.

Отказ

Наблюдаемое отклонение поведения системы от ожидаемого при её выполнении. Синоним — failure.

Цепочка: **ошибка человека, дефект в артефакте, отказ при выполнении**. Непосредственно наблюдать можно только отказ; дефект устанавливается при анализе его причины.

Дефект: классификация

Основание	Категории
Этап возникновения	Требования · проектирование · реализация · конфигурация окружения · данные
Природа	Функциональный · нефункциональный (производительность, безопасность, надёжность)
Область проявления	Логика · интерфейс · интеграция · данные · документация
Степень ущерба	От блокирующего до косметического; шкала severity
Воспроизводимость	Постоянный · периодический · зависит от окружения или данных
Источник	Требование · код · конфигурация · внешняя зависимость

Классификация по воспроизводимости определяет форму отчёта: для периодического дефекта в отчёт добавляются условия воспроизведения и частота проявления.

Дефект: жизненный цикл и стоимость обнаружения

Жизненный цикл

Обнаружен — дефект зафиксирован в отчёте.

В работе — устанавливается причина, вносится исправление.

Исправлен — изменение внесено в код.

Проверен — устранение подтверждено на новой сборке.

Закрит — либо **отклонён**, если дефектом не признан.

Стоимость исправления по этапам

Требования — правка формулировки.

Проектирование — пересмотр решения.

Реализация — изменение кода и повторный прогон.

Эксплуатация — изменение кода, повторная поставка, последствия для данных.

Стоимость растёт с каждым этапом: чем позже обнаружен дефект, тем больше артефактов и данных успело от него зависеть. После исправления проверяется и **регресс**: не нарушена ли работавшая ранее функциональность.

МОДУЛЬ 2 · ЛЕКЦИЯ 4

Баг-репорты

Баг-репорт: определение и назначение

Баг-репорт (defect report) — документ, фиксирующий обнаруженный дефект: условия воспроизведения, ожидаемый и фактический результат, оценку влияния (ISTQB CTFL 6.6.2; STF 6.6).

Читатели отчёта

Разработчик — воспроизводит дефект и находит причину: требуются точные шаги и данные.

Тест-менеджер — планирует исправления: требуются severity и priority.

Руководство проекта — оценивает риски: требуется влияние на пользователя.

Требования к отчёту

Воспроизводимость — дефект повторяется по шагам отчёта.

Локализуемость — ясно, какой объект и какое правило затронуты.

Обоснованность — ожидаемый результат опирается на требование.

Некачественный отчёт затягивает исправление: возвращается с уточняющими вопросами или откладывается. По Канеру, отчёт — утверждение, что продукт стал менее ценным для некоторого круга пользователей.

Баг-репорты: структура отчёта

Обязательное ядро

Заголовок (что, где, при каких условиях)

Шаги воспроизведения (Steps to Reproduce)

Ожидаемый результат

Фактический результат

Специфичность STR: детали важнее — иначе не воспроизвести

Типовые атрибуты (STF Table 6-1)

ID, объект теста, версия, платформа, дата, статус

Severity, priority

Связанное требование, тест-кейс

Источник проблемы, комментарии, ссылки

Вложения: скриншоты/видео, файлы; поля Jira: Summary, Priority, Environment, Description, STR

Сквозной пример: плохой баг-репорт

Заголовок: «Не работает создание клиента»

Шаги воспроизведения	1. Отправить запрос на создание клиента 2. Получить ошибку
Ожидаемый результат	Клиент создан
Фактический результат	Ошибка
Severity / Priority	—

Нет данных запроса: какие поля, какие значения.

Не указан код и тело ответа.

Не указаны severity и priority.

Сквозной пример: хороший баг-репорт

BR-001: Создание клиента возвращает 201 при email длиной 255 символов вместо 400 VALIDATION_ERROR

Шаги воспроизведения	1. POST /api/v1/clients/create-individual 2. Заполнить contacts.email строкой из 255 символов (см. вложение) 3. Остальные поля — валидные значения (см. вложение)
Ожидаемый результат	400, код VALIDATION_ERROR, клиент не создан
Фактический результат	201 Created, клиент создан
Связанное требование	email длиной не более 254 символов
Severity / Priority	SERIOUS (данные вне контракта попадают в систему) / NEXT RELEASE

Реальный пример из практики

Место для реального примера. Здесь на занятии разбирается действительный баг-репорт из рабочей практики: заголовок, шаги воспроизведения, ожидаемый и фактический результат, severity и priority.

На что смотрим в реальном примере

Как сформулирован заголовок.

Насколько специфичны шаги.

Как обоснованы severity и priority.

Какие атрибуты используются в реальном трекере.

Что обсудить со студентами

Почему дефект классифицирован именно так.

Что помогло воспроизвести проблему.

Как отчёт повлиял на приоритет исправления.

Что бы вы улучшили в формулировках.

Баг-репорты: severity vs priority

Severity (важность)

Степень ущерба от дефекта — с точки зрения пользователя.

Не меняется без новых данных о дефекте.

Классы (STF): 1 FATAL, 2 VERY SERIOUS, 3 SERIOUS, 4 MODERATE, 5 MILD.

Priority (срочность)

Как быстро исправить — зависит от контекста проекта.

Может меняться: договорённости, релиз, заказчик.

Классы (STF): 1 IMMEDIATE, 2 NEXT RELEASE, 3 ON OCCASION, 4 OPEN.

Severity $\neq$ Priority: серьёзный дефект может иметь низкий приоритет (и наоборот — косметический с высоким). Пример: ошибка расчёта дат до 1999 г. против перевёрнутого логотипа.

Баг-репорты: типичные ошибки и стандарты

Типичные ошибки

Расплывчатый заголовок.

Отсутствие важных шагов или лишние шаги.

«Решение вместо проблемы».

Несколько дефектов в одном отчёте.

Стандарты классификации

IEEE 1044-2009 — классификация аномалий (формально отозван в 2020).

Актуальная линейка — ISO/IEC/IEEE 29119; шаблоны документации — IEEE 829 / 29119-3.

На практике шаблон задаёт тест-менеджер единым для проекта.

Единый шаблон баг-репорта курса

Поле	Что указывать
ID	Уникальный идентификатор, например BR-001
Заголовок	Что, где и при каком условии — одной строкой
Шаги воспроизведения	Нумерованные, со конкретными данными и endpoint
Ожидаемый результат	Код ответа, тело, состояние системы
Фактический результат	Что произошло в действительности
Severity	Степень ущерба по шкале
Priority	Срочность исправления по шкале
Связанное требование	Требование или тест-кейс, который нарушен

Резюме лекции 4

Таблицы решений: правила = тест-кейсы; каждый столбец — комбинация условий; число правил полной таблицы — 2^n ; для тестов — несхлопнутая таблица.

State-transition: покрытие всех переходов; таблицы переходов выявляют пробелы в требованиях; применяется, когда важен порядок операций.

Дефект: цепочка «ошибка → дефект → отказ»; классификация по этапу возникновения, природе, воспроизводимости; стоимость исправления растёт с этапом обнаружения.

Баг-репорт: заголовок + STR + expected + actual + severity + priority; практика — в модуле 7.

Источники

ISTQB — Certified Tester Foundation Level (CTFL) v4.0, Syllabus и Glossary.

Copeland, Lee. *A Practitioner's Guide to Software Test Design*. Artech House.

Куликов, С. С. *Тестирование программного обеспечения*. Базовый курс.

Kaner, Bach, Pettichord. *Lessons Learned in Software Testing* (LLST / TTS).

ISO/IEC/IEEE 29119 — документация и техники тестирования; исторический ориентир — IEEE 829.

IEEE 1044-2009 — классификация аномалий (переведён в статус неактивного в 2020).



Вопросы?